UT3 Bloque 5 CRUD web completo con Servlets JSP y MVC

Material de estudio y guía de prácticas

En este bloque convertirás interfaces comerciales ya construidas en aplicaciones dinámicas. No tendrás que diseñar tarjetas, grids ni estilos responsive: tu trabajo será conectar Oracle, JDBC, DAO, métodos privados de mapeo, objetos, controllers y JSP.

Meta de aprendizaje. Al terminar podrás implementar un CRUD completo y explicar el recorrido de cualquier dato desde una tabla hasta el navegador y desde un formulario hasta Oracle.

Etapa	Aplicación	Reto principal
1	BurgerLab	Leer listado y detalle
2	TripNova	Crear registros
3	CineVerse	Editar registros
4	ByteMarket	Eliminar por POST y reunir CRUD
Final	StayNest	Integración con mayor autonomía

Cómo utilizar los materiales

1. Instala primero la base de datos de la práctica con 01_schema.sql y 02_data.sql.
2. Importa la carpeta de trabajo como proyecto Maven y configura Java 17 y Tomcat 10.1.
3. Lee el objetivo de cada TODO antes de modificar el archivo.
4. Mantén el HTML y el CSS proporcionados; completa únicamente la funcionalidad indicada.
5. Después de cada cambio, ejecuta una prueba concreta y reconstruye el flujo de la petición.

Del caso de uso a las capas

```
Usuario pide una URL o envía un formulario
        -> Tomcat localiza el Servlet
        -> el controller decide el caso de uso
        -> DAO ejecuta SQL con JDBC
        -> métodos privados de mapeo convierte ResultSet en objetos
        -> controller publica atributos
        -> JSP usa EL y JSTL
        -> Tomcat devuelve HTML
        -> el navegador dibuja la interfaz
```

Qué hace cada pieza

Pieza	Responsabilidad	No debe hacer
Controller	Leer HTTP, validar, coordinar y elegir vista	Escribir una interfaz completa
DAO	Ejecutar SQL y ocultar JDBC al resto	Conocer botones o CSS
métodos privados de mapeo	Convertir una fila en objetos	Decidir navegación
Model	Representar datos y relaciones	Abrir conexiones
JSP	Representar datos ya preparados	Consultar Oracle

CRUD SQL y HTTP

Operación	SQL	HTTP del bloque
Crear	INSERT	POST
Leer	SELECT	GET
Editar	UPDATE	POST
Eliminar	DELETE	POST

GET se utiliza para consultar y mostrar. POST se utiliza cuando la petición cambia datos. En este bloque no usamos PUT, DELETE HTTP ni una arquitectura REST.

Controller organizado por acciones

```
String accion = request.getParameter("accion");
if (accion == null) accion = "listar";

switch (accion) {
    case "listar" -> listar(request, response);
    case "detalle" -> detalle(request, response);
    default -> response.sendError(400);
}
```

El parámetro accion llega desde una URL o un formulario. El switch no implementa el caso de uso: solo dirige la petición al método adecuado. Cada método prepara exactamente los datos que necesita la pantalla.

Campos hidden

```
<input type="hidden" name="accion" value="actualizar">
<input type="hidden" name="id" value="${pedido.id}">
```

El campo hidden viaja en el formulario aunque no se muestre. En una edición, el id permite construir el WHERE correcto. En una eliminación, accion indica qué rama de doPost debe ejecutarse.

Post Redirect Get

```
POST -> DAO escribe -> sendRedirect -> GET -> listado actualizado
```

Después de escribir en Oracle, el controller no realiza forward al listado. Ordena al navegador hacer una petición GET nueva. De ese modo, F5 repite una lectura y no la escritura anterior.

Las imágenes también son datos

```
Oracle guarda /assets/img/catalogo/foto.webp
  -> métodos privados de mapeo lee la columna
  -> el objeto conserva la ruta
  -> controller transporta el objeto
  -> JSP construye src con el context path
  -> el navegador solicita el archivo local
```

No guardamos la fotografía dentro de Oracle. Guardamos una ruta de texto. El archivo WebP se encuentra dentro de la aplicación web y Tomcat lo sirve como recurso estático.

Cómo diagnosticar una imagen rota

1. Consulta la columna imagen directamente en Oracle.
2. Comprueba que el métodos privados de mapeo llama al setter de la propiedad correcta.
3. Inspecciona el objeto antes de publicarlo en request.
4. Mira el atributo src final con las herramientas del navegador.
5. Abre Network y confirma la URL y el código de respuesta.
6. Comprueba que el archivo existe con el mismo nombre y extensión.

Práctica 1 BurgerLab

READ completo: listado, detalle, objetos relacionados, imágenes y flujo Oracle a navegador.

Dato	Contenido
Entidad principal	Producto
Ruta	/productos
Registros	18
Relaciones	Categoria
Fotografías locales	18

Resultado que debes obtener

- Catálogo responsive con tarjetas completas y estado vacío.
- Detalle accesible mediante un id dinámico.
- Fotografías locales cargadas desde la ruta almacenada en Oracle.

Peticiones que intervienen

Petición	Qué debe preparar	Qué debes observar
GET /productos	listar	SELECT y listado.jsp
GET /productos?accion=detalle&id=...	detalle	findById y detalle.jsp

Pasos de trabajo

1. Ejecuta los scripts Oracle y verifica las filas de la entidad principal y de sus tablas relacionadas.
2. Localiza Model, métodos privados de mapeo, DAO, DAO concreto, Controller y WEB-INF/views.
3. Completa primero la lectura; inspecciona los objetos antes de abrir la JSP.
4. Continúa con los TODO de la vista sin alterar las clases CSS ni la composición.
5. Prueba una colección vacía y varios valores de id.

TODO de las JSP

TODO	Objetivo	Pista
1	Variable del recorrido	Da un nombre singular al elemento actual.
2	Colección	Utiliza el atributo publicado por el controller.
3	Fotografía	Lee la propiedad que contiene la ruta local.
4	Texto alternativo y título	Usa una propiedad comprensible del elemento actual.
5	Descripción	Muestra el texto largo del modelo.
6	Distintivo	Navega por el objeto relacionado cuando corresponda.
7	Metadato	Muestra el dato secundario indicado en la interfaz.
8	Enlace	Incluye el identificador del elemento actual.

TODO	Objetivo	Pista
9	Importe	Representa el valor numérico si la práctica lo utiliza.

Pruebas obligatorias

- Listado con datos y sin datos.
- Detalle correcto, id inexistente e id no numérico.
- Todas las imágenes responden y su src contiene el context path una sola vez.

Cierre de la práctica. Elige una tarjeta y explica de qué tabla y columna procede cada texto, qué método DAO lo recuperó, qué objeto lo contiene y qué expresión lo convirtió en HTML.

Práctica 2 TripNova

READ y CREATE: formulario GET, inserción POST, validación mínima y Post Redirect Get.

Dato	Contenido
Entidad principal	Destino
Ruta	/destinos
Registros	15
Relaciones	TipoDestino
Fotografías locales	15

Resultado que debes obtener

- Catálogo responsive con tarjetas completas y estado vacío.
- Detalle accesible mediante un id dinámico.
- Fotografías locales cargadas desde la ruta almacenada en Oracle.
- Formulario de alta que inserta y termina con redirect.

Peticiones que intervienen

Petición	Qué debe preparar	Qué debes observar
GET /destinos	listar	SELECT y listado.jsp
GET /destinos?accion=detalle&id=...	detalle	findById y detalle.jsp
GET /destinos?accion=nuevo	mostrarNuevo	Prepara relaciones y formulario
POST /destinos accion=crear	crear	INSERT y redirect

Pasos de trabajo

1. Ejecuta los scripts Oracle y verifica las filas de la entidad principal y de sus tablas relacionadas.
2. Localiza Model, métodos privados de mapeo, DAO, DAO concreto, Controller y WEB-INF/views.
3. Completa primero la lectura; inspecciona los objetos antes de abrir la JSP.
4. Continúa con los TODO de la vista sin alterar las clases CSS ni la composición.
5. Prueba una colección vacía y varios valores de id.
6. Completa GET del formulario, POST de alta, INSERT y redirect.

TODO de las JSP

TODO	Objetivo	Pista
1	Variable del recorrido	Da un nombre singular al elemento actual.
2	Colección	Utiliza el atributo publicado por el controller.
3	Fotografía	Lee la propiedad que contiene la ruta local.
4	Texto alternativo y título	Usa una propiedad comprensible del elemento actual.

TODO	Objetivo	Pista
5	Descripción	Muestra el texto largo del modelo.
6	Distintivo	Navega por el objeto relacionado cuando corresponda.
7	Metadato	Muestra el dato secundario indicado en la interfaz.
8	Enlace	Incluye el identificador del elemento actual.
9	Importe	Representa el valor numérico si la práctica lo utiliza.

TODO del formulario

- Construye action con el context path y la ruta del controller.
- Selecciona el método HTTP que modifica datos.
- Envía accion e id mediante campos ocultos cuando proceda.
- Haz coincidir cada name con el getParameter que lo leerá.
- Rellena los select recorriendo las colecciones preparadas por el controller.
- Muestra un nombre comprensible pero envía el identificador como value.

Pruebas obligatorias

- Listado con datos y sin datos.
- Detalle correcto, id inexistente e id no numérico.
- Todas las imágenes responden y su src contiene el context path una sola vez.
- Alta correcta y alta con un campo obligatorio ausente.

Cierre de la práctica. Elige una tarjeta y explica de qué tabla y columna procede cada texto, qué método DAO lo recuperó, qué objeto lo contiene y qué expresión lo convirtió en HTML.

Práctica 3 CineVerse

UPDATE: cargar el registro, reutilizar formulario, conservar ID y actualizar con POST.

Dato	Contenido
Entidad principal	Pelicula
Ruta	/peliculas
Registros	15
Relaciones	Genero
Fotografías locales	15

Resultado que debes obtener

- Catálogo responsive con tarjetas completas y estado vacío.
- Detalle accesible mediante un id dinámico.
- Fotografías locales cargadas desde la ruta almacenada en Oracle.
- Formulario de edición cargado con el registro solicitado.

Peticiones que intervienen

Petición	Qué debe preparar	Qué debes observar
GET /peliculas	listar	SELECT y listado.jsp
GET /peliculas?accion=detalle&id=...	detalle	findById y detalle.jsp
GET /peliculas?accion=editar&id=...	mostrarEditar	Carga objeto y formulario
POST /peliculas accion=actualizar	actualizar	UPDATE y redirect

Pasos de trabajo

1. Ejecuta los scripts Oracle y verifica las filas de la entidad principal y de sus tablas relacionadas.
2. Localiza Model, métodos privados de mapeo, DAO, DAO concreto, Controller y WEB-INF/views.
3. Completa primero la lectura; inspecciona los objetos antes de abrir la JSP.
4. Continúa con los TODO de la vista sin alterar las clases CSS ni la composición.
5. Prueba una colección vacía y varios valores de id.
6. Carga el objeto en GET, conserva el id y ejecuta UPDATE desde POST.

TODO de las JSP

TODO	Objetivo	Pista
1	Variable del recorrido	Da un nombre singular al elemento actual.
2	Colección	Utiliza el atributo publicado por el controller.
3	Fotografía	Lee la propiedad que contiene la ruta local.

TODO	Objetivo	Pista
4	Texto alternativo y título	Usa una propiedad comprensible del elemento actual.
5	Descripción	Muestra el texto largo del modelo.
6	Distintivo	Navega por el objeto relacionado cuando corresponda.
7	Metadato	Muestra el dato secundario indicado en la interfaz.
8	Enlace	Incluye el identificador del elemento actual.
9	Importe	Representa el valor numérico si la práctica lo utiliza.

TODO del formulario

- Construye action con el context path y la ruta del controller.
- Selecciona el método HTTP que modifica datos.
- Envía accion e id mediante campos ocultos cuando proceda.
- Haz coincidir cada name con el getParameter que lo leerá.
- Rellena los select recorriendo las colecciones preparadas por el controller.
- Muestra un nombre comprensible pero envía el identificador como value.

Pruebas obligatorias

- Listado con datos y sin datos.
- Detalle correcto, id inexistente e id no numérico.
- Todas las imágenes responden y su src contiene el context path una sola vez.
- Edición de texto y cambio de objeto relacionado.

Cierre de la práctica. Elige una tarjeta y explica de qué tabla y columna procede cada texto, qué método DAO lo recuperó, qué objeto lo contiene y qué expresión lo convirtió en HTML.

Práctica 4 ByteMarket

CRUD completo con especial atención a DELETE mediante POST y restricciones de integridad.

Dato	Contenido
Entidad principal	Producto
Ruta	/productos
Registros	20
Relaciones	Marca, Categoria
Fotografías locales	20

Resultado que debes obtener

- Catálogo responsive con tarjetas completas y estado vacío.
- Detalle accesible mediante un id dinámico.
- Fotografías locales cargadas desde la ruta almacenada en Oracle.
- Formulario de alta que inserta y termina con redirect.
- Formulario de edición cargado con el registro solicitado.
- Eliminación enviada exclusivamente por POST.

Peticiones que intervienen

Petición	Qué debe preparar	Qué debes observar
GET /productos	listar	SELECT y listado.jsp
GET /productos?accion=detalle&id=...	detalle	findById y detalle.jsp
GET /productos?accion=nuevo	mostrarNuevo	Prepara relaciones y formulario
POST /productos accion=crear	crear	INSERT y redirect
GET /productos?accion=editar&id=...	mostrarEditar	Carga objeto y formulario
POST /productos accion=actualizar	actualizar	UPDATE y redirect
POST /productos accion=eliminar	eliminar	DELETE y redirect

Pasos de trabajo

1. Ejecuta los scripts Oracle y verifica las filas de la entidad principal y de sus tablas relacionadas.
2. Localiza Model, métodos privados de mapeo, DAO, DAO concreto, Controller y WEB-INF/views.
3. Completa primero la lectura; inspecciona los objetos antes de abrir la JSP.
4. Continúa con los TODO de la vista sin alterar las clases CSS ni la composición.
5. Prueba una colección vacía y varios valores de id.
6. Completa GET del formulario, POST de alta, INSERT y redirect.
7. Carga el objeto en GET, conserva el id y ejecuta UPDATE desde POST.

8. Envía accion e id con hidden, elimina por POST y vuelve al listado.

TODO de las JSP

TODO	Objetivo	Pista
1	Variable del recorrido	Da un nombre singular al elemento actual.
2	Colección	Utiliza el atributo publicado por el controller.
3	Fotografía	Lee la propiedad que contiene la ruta local.
4	Texto alternativo y título	Usa una propiedad comprensible del elemento actual.
5	Descripción	Muestra el texto largo del modelo.
6	Distintivo	Navega por el objeto relacionado cuando corresponda.
7	Metadato	Muestra el dato secundario indicado en la interfaz.
8	Enlace	Incluye el identificador del elemento actual.
9	Importe	Representa el valor numérico si la práctica lo utiliza.

TODO del formulario

- Construye action con el context path y la ruta del controller.
- Selecciona el método HTTP que modifica datos.
- Envía accion e id mediante campos ocultos cuando proceda.
- Haz coincidir cada name con el getParameter que lo leerá.
- Rellena los select recorriendo las colecciones preparadas por el controller.
- Muestra un nombre comprensible pero envía el identificador como value.

Pruebas obligatorias

- Listado con datos y sin datos.
- Detalle correcto, id inexistente e id no numérico.
- Todas las imágenes responden y su src contiene el context path una sola vez.
- Alta correcta y alta con un campo obligatorio ausente.
- Edición de texto y cambio de objeto relacionado.
- Eliminación por POST y comportamiento ante una restricción FK.

Cierre de la práctica. Elige una tarjeta y explica de qué tabla y columna procede cada texto, qué método DAO lo recuperó, qué objeto lo contiene y qué expresión lo convirtió en HTML.

Práctica final StayNest

Integración autónoma de CRUD, dos relaciones, métodos privados de mapeo, DAO, MVC, imágenes y pruebas completas.

Dato	Contenido
Entidad principal	Alojamiento
Ruta	/alojamientos
Registros	18
Relaciones	Localidad, Tipo
Fotografías locales	18

Resultado que debes obtener

- Catálogo responsive con tarjetas completas y estado vacío.
- Detalle accesible mediante un id dinámico.
- Fotografías locales cargadas desde la ruta almacenada en Oracle.
- Formulario de alta que inserta y termina con redirect.
- Formulario de edición cargado con el registro solicitado.
- Eliminación enviada exclusivamente por POST.

Peticiones que intervienen

Petición	Qué debe preparar	Qué debes observar
GET /alojamientos	listar	SELECT y listado.jsp
GET /alojamientos?accion=detalle&id=...	detalle	findById y detalle.jsp
GET /alojamientos?accion=nuevo	mostrarNuevo	Prepara relaciones y formulario
POST /alojamientos accion=crear	crear	INSERT y redirect
GET /alojamientos?accion=editar&id=...	mostrarEditar	Carga objeto y formulario
POST /alojamientos accion=actualizar	actualizar	UPDATE y redirect
POST /alojamientos accion=eliminar	eliminar	DELETE y redirect

Pasos de trabajo

1. Ejecuta los scripts Oracle y verifica las filas de la entidad principal y de sus tablas relacionadas.
2. Localiza Model, métodos privados de mapeo, DAO, DAO concreto, Controller y WEB-INF/views.
3. Completa primero la lectura; inspecciona los objetos antes de abrir la JSP.
4. Continúa con los TODO de la vista sin alterar las clases CSS ni la composición.
5. Prueba una colección vacía y varios valores de id.
6. Completa GET del formulario, POST de alta, INSERT y redirect.

7. Carga el objeto en GET, conserva el id y ejecuta UPDATE desde POST.
8. Envía accion e id con hidden, elimina por POST y vuelve al listado.

TODO de las JSP

TODO	Objetivo	Pista
1	Variable del recorrido	Da un nombre singular al elemento actual.
2	Colección	Utiliza el atributo publicado por el controller.
3	Fotografía	Lee la propiedad que contiene la ruta local.
4	Texto alternativo y título	Usa una propiedad comprensible del elemento actual.
5	Descripción	Muestra el texto largo del modelo.
6	Distintivo	Navega por el objeto relacionado cuando corresponda.
7	Metadato	Muestra el dato secundario indicado en la interfaz.
8	Enlace	Incluye el identificador del elemento actual.
9	Importe	Representa el valor numérico si la práctica lo utiliza.

TODO del formulario

- Construye action con el context path y la ruta del controller.
- Selecciona el método HTTP que modifica datos.
- Envía accion e id mediante campos ocultos cuando proceda.
- Haz coincidir cada name con el getParameter que lo leerá.
- Rellena los select recorriendo las colecciones preparadas por el controller.
- Muestra un nombre comprensible pero envía el identificador como value.

Pruebas obligatorias

- Listado con datos y sin datos.
- Detalle correcto, id inexistente e id no numérico.
- Todas las imágenes responden y su src contiene el context path una sola vez.
- Alta correcta y alta con un campo obligatorio ausente.
- Edición de texto y cambio de objeto relacionado.
- Eliminación por POST y comportamiento ante una restricción FK.

Cierre de la práctica. Elige una tarjeta y explica de qué tabla y columna procede cada texto, qué método DAO lo recuperó, qué objeto lo contiene y qué expresión lo convirtió en HTML.

Errores frecuentes y pistas de diagnóstico

Síntoma	Qué debes comprobar	Causa probable
accion es null	Revisa URL o Form Data	Falta el parámetro que dirige el switch
NumberFormatException	Mira el valor exacto de id	El dato está vacío o no es numérico
DAO no inicializado	Comprueba init	La referencia nunca recibió un DAO concreto
Detalle vacío	Ejecuta findById en aislamiento	No existe la fila o el WHERE es incorrecto
EL no muestra nada	Compara la clave y la propiedad	El contrato controller vista no coincide
POST recibe null	Inspecciona name y getParameter	Los nombres son distintos
UPDATE no afecta filas	Comprueba el id hidden	El WHERE recibe un id ausente
DELETE bloqueado	Lee la FK indicada por Oracle	Existe un registro dependiente
Redirect 404	Mira Location y context path	La URL final está mal construida
Imagen rota	Sigue Oracle métodos privados de mapeo objeto EL src Network	La cadena se rompe en uno de los pasos

Mapa mental final

```
LECTURA
Oracle -> DAO -> métodos privados de mapeo -> Model -> Controller -> request -> JSP -> HTML -> navegador

FORMULARIO
navegador -> GET o POST -> Controller -> Model -> DAO -> Oracle

DESPUÉS DE ESCRIBIR
POST -> INSERT UPDATE DELETE -> redirect -> GET -> representación actualizada
```

Comprueba que puedes explicar

- Por qué READ utiliza GET y las escrituras utilizan POST.
- Qué diferencia existe entre parámetro HTTP y atributo de request.
- Cómo se convierte una fila con relaciones en un objeto completo.
- Para qué sirven accion e id dentro de campos hidden.
- Por qué el redirect evita repetir una escritura con F5.
- Por qué una eliminación no debe realizarse mediante un enlace GET.
- Cómo diagnosticar una imagen desde Oracle hasta Network.
- Qué responsabilidad corresponde a cada componente MVC.